

Constellation Project Week 6 Report

Team 3

Aidan McGrath, Marlonn Dubreuil, and Anirudh Ganesan

I. INTRODUCTION AND PROJECT PURPOSE

Accurate electrical load forecasting is important for energy providers because expected demand affects generation planning, energy purchasing, infrastructure utilization, and overall grid operations. This project develops a machine-learning model for Constellation that predicts electrical load approximately one month in advance for customers in the Newark, New Jersey area. Load is reported separately by rate class, including Residential, several commercial and industrial General Service classes, and the CIEP and RSCP classes.

This report covers Week 6. The proposal set one milestone for this week: a functioning decision-tree model trained on at least the weather features, evaluated on held-out historical data, and saved with Python's pickle. That milestone is met. A decision tree for Residential load, trained on Newark weather and calendar features, reaches 11.5% mean absolute percentage error (MAPE) on a 30-day held-out test, which is the closest match to one month ahead. The saved model reloads and reproduces its predictions exactly. This result assumes the weather that actually occurred is known. When we replace it with typical weather for the month and hour, error rises by 1.6 to 2.9 percentage points of weighted error (WAPE), which shows how much accuracy depends on weather knowledge at prediction time (Section VI). The team repository also holds a weather-only LSTM, which we ran for comparison.

II. PROJECT BENEFICIARIES

The primary beneficiary is Constellation and its data analytics and energy-planning teams, who could use more accurate forecasts when estimating demand and planning energy procurement. Grid and operations staff and business planners benefit indirectly. Our early results support the focus on rate classes: Residential load is driven mainly by temperature, while a commercial and industrial class (GTI) follows the work calendar, so class-specific forecasts give each group different information.

III. LITERATURE REVIEW

We searched Google Scholar for work on neural networks, recurrent networks, tree-based models, weather effects, and COVID-19 in load forecasting. Short-term load forecasting is a long-studied problem. Hippert et al. reviewed 40 papers that applied neural networks to it [1], and Taylor and McSharry compared forecasting methods on European data [2].

Recurrent and hybrid networks. Kong et al. applied an LSTM, a recurrent network with memory, to residential load using public smart-meter data [3]. Abumohsen et al. compared RNN, LSTM, and GRU models on short-term loads in Palestine [4].

TABLE 1: Data Sources

Source	Coverage	Used
Hourly load, 9 classes (Res, GPC, GPI, GSC, GSI, GSTC, GSTI, GTC, GTI), MW	May 2020 to May 2024	Yes
Hourly load, CIEP and RSCP, MW	Jul 2001 to May 2025	No
Newark (KEWR) hourly weather	2019 to 2024	Yes
Calendar flags (weekday, weekend, month, holiday)	Jan 2019 to Sep 2024	Yes
NJ economic series (BEA, BLS)	Varies	No
NJ energy series (EIA), monthly	Varies	No

Hybrids pair convolutional layers, which find local patterns in a window of hours, with recurrent layers, which carry longer memory; examples are Alhussein et al. [5], Eskandari et al. [6], and a review with simulations by Ullah et al. [7]. This is the design behind our planned CRNN stretch model, and the plain LSTM in the team repository is the first step toward it.

Tree-based models. Dudek studied random forests for short-term load forecasting [8]. Moon et al. compared tree ensembles, including random forest and gradient boosting, on building loads and used Shapley values to explain which inputs mattered [9]. These support our plan to move from a single tree to ensembles while keeping feature importance.

Weather and calendar features. Asbury's weather-load model uses a threshold temperature for air conditioning [10], the pattern our trees are meant to capture. Imani built a load-temperature CNN for residential load [11]. Wang et al. fed weather and workday/holiday features into an LSTM for residential forecasts [12]. Ijaz et al. [13] and Yu et al. [14] address feature selection; Yu et al. use Spearman rank correlation, which matches our rank-based screening idea.

COVID-19. Alasali et al. document how the pandemic changed load behavior and forecasting [15]. Obst et al. built adaptive methods for forecasting during and after the French lockdown [16]. We instead exclude the affected months, which is simpler but discards data; the adaptive approach is an alternative if the exclusion costs too much accuracy.

Most recurrent-network studies above use household or building data. Our targets are utility-level loads by rate class, so household results may not transfer directly. **[Team: these summaries come from Google Scholar titles and snippets; read your assigned papers and correct any claim.]**

IV. DATA AND PREPROCESSING

The data comes from a course-provided package [17]. Table 1 lists what it holds and what we have used.

The proposal expected trouble aligning data in different formats, and that proved true. Four issues affected the work.

1. *Mixed weather timestamp formats.* The 2019, 2022, and 2023 weather files use dates like 2019-01-01 1:51 AM; the 2020, 2021, and 2024 files use 1/1/2020 1:51 in 24-hour time. Parsing all files with one inferred format silently dropped three years of weather. The first run lost 8,126 rows and its error got worse (21% MAPE). Parsing each file separately fixed it; 66 rows are now dropped.
2. *Daylight saving time.* Raw load files carry an extra “Hour 25” for the repeated fall-back hour. We resample to a strict hourly index, averaging duplicates and interpolating the missing spring-forward hour.
3. *Irregular weather timestamps.* Readings arrive at times like :51 past the hour. We average them to the top of the hour and fill gaps up to six hours by interpolation.
4. *COVID-19 period.* See Section V.

The proposal planned to use the NCSA Delta supercomputer because of dataset size. So far the data fit easily in our workspace: a Residential tree fits in about 0.05 seconds and the LSTM trains in about two minutes on two CPU threads. Delta may be needed for all-class runs and the CRNN.

V. METHODS

A. Decision-tree pipeline

One command builds the dataset and runs the models for any rate class. The code is split into loaders, feature building, model training, and a command-line runner, and it saves cleaned data and models as pickle files. The pipeline lives in the team repository (github.com/McGrath765353/DS340W—Team-3) under `src/`, next to the LSTM script. **[Team: copy the `src/` folder from github.com/MarlonnD718/jcpl-load-forecasting into the team repository.]**

Features (19). Temperature, dew point, humidity, a weekday flag, a weekend flag, 12 month flags, a holiday flag, and hour of day. Weather is joined to load on the hourly timestamp; daily calendar flags are repeated across each day’s 24 hours.

COVID-19 exclusion. We drop 1 March 2020 to 30 June 2021, because lockdown-era demand does not reflect how the market behaves now. For Residential this removes 9,458 hourly rows and leaves 25,540 (about 2.9 years). These dates are a working assumption the group has not confirmed.

Model. One scikit-learn decision tree regressor [18] per rate class, with maximum depth 8 and at least 20 samples per leaf. These settings are untuned starting values.

Evaluation. Splits are chronological, never random. For each horizon of 7, 15, or 30 days, we train on everything before the final h days and test on those days. The 30-day test is the closest match to the proposal’s one-month target. We report MAE and RMSE in MW, and MAPE (the average percentage miss). Where we compare with the LSTM we also use WAPE, total absolute error divided by total actual load, which weights busy hours more.

TABLE 2: Residential Decision Tree by Horizon (Observed Weather)

Horizon	Hours	MAE	RMSE	MAPE	MAPE, no wx
7 days	168	205.8	294.3	14.5%	16.1%
15 days	360	162.5	236.7	11.9%	16.5%
30 days	720	142.7	214.1	11.5%	13.9%
Average				12.6%	15.5%

MAE and RMSE in MW. “No wx” = calendar flags only.

TABLE 3: Residential, 25 Oct 2023 to 31 May 2024

Model	MAE	RMSE	WAPE	WAPE
	obs. wx	obs. wx	obs. wx	typ. wx
Weekday-hour average	214.6	264.2	18.0%	—
LSTM, weather only	178.3	249.8	15.0%	15.7%
Tree, calendar only	191.9	259.2	16.2%	—
Tree, weather + hour	169.7	235.2	14.3%	16.1%
Tree, all 19 inputs	171.0	239.0	14.4%	16.0%

MAE and RMSE in MW. Obs. = observed weather; typ. = average weather for the month and hour.

B. Weather-only LSTM (team repository)

The team repository holds `newark_load_single.py`, a PyTorch [19] script that trains one LSTM per rate class. Each prediction uses the previous 48 hours of seven weather readings (temperature, dew point, humidity, wind speed, wind gust, pressure, precipitation) and no calendar or past-load inputs. It splits the series in time (70% train, 15% validation for early stopping, 15% test), scales inputs with training data only, and reports MAE, RMSE, and WAPE. It scores two weather scenarios, observed weather and training-period average weather for each month and hour, and a weekday-by-hour average-load baseline. It does not exclude COVID months. We ran it unchanged on Residential.

VI. RESULTS

Decision tree, Residential. Adding weather cut average MAPE from 15.5% to 12.6% (Table 2). Error fell at all three horizons.

Temperature carries 71% to 72% of the tree’s importance at every horizon and hour of day about 22%, together roughly 93% to 94%. Every other input, including dew point and the weekday, holiday, and month flags, is at or below 1.4%. Temperature is the first predictor and the time-of-day pattern the second; the weekday, holiday, and month flags add little for Residential.

A second class behaves differently. For GTI, a commercial and industrial class, average MAPE is 31.1% (31.9%, 25.8%, 35.6% at 7, 15, and 30 days). The weekday flag alone carries about 65% of importance, followed by the holiday flag, dew point, hour, and temperature at roughly 5% to 8% each. Business load follows the work calendar more than the weather, so one model setting will not suit every class.

Tree and LSTM on the same period. Table 3 scores both models on the LSTM’s test period, 25 October 2023 to 31 May 2024 (about 5,200 hours), under observed weather and under typical weather. The proposal asked what weather is realistically available at prediction time; the typical-weather column is our first answer.

TABLE 4: Status Against the Proposal

Proposal item	Status
Clean and match load and Newark weather	Done
Decision tree trained on weather features (Week 6)	Done
Chronological held-out evaluation	Done
Pickle the model and load it later (Week 6)	Done; reload gives identical predictions
Add customer counts (Weeks 6–9)	Not started
Compare against weather-only baseline (Week 9)	Planned
Add solar, gas, coal series	Future expansion
Meeting with Daniel Walker	[Team: report outcome]

Both models beat the weekday-by-hour average. With observed weather the tree is ahead by 0.6 points of WAPE; with typical weather the two are within 0.4 points and the LSTM is slightly ahead. Each model was run once, so we do not call a winner. The tree does as well with weather and hour alone as with all 19 inputs. On the 30-day test, the tree’s WAPE rises from 13.0% to 15.9% when typical weather replaces observed weather.

The comparison is not perfectly even. The tree trains with COVID months removed (20,294 hours), while the LSTM trains on the first 70% of the series, which includes them. The LSTM scores 5,199 hours because it needs 48 hours of weather before each prediction; the tree scores 5,246. Error also does not shrink as the horizon grows: the three tree test windows are nested and end on 31 May 2024, so differences reflect which days were included, not forecasting difficulty.

VII. PROGRESS, LIMITATIONS, AND NEXT STEPS

A. Progress against the proposal

Table 4 compares the proposal’s plan with the current state.

The proposal also names solar, natural gas, and coal generation. We have scoped these as future expansion after the initial model is complete. The energy files provided cover solar (small-scale and utility-scale), wind, and biomass, plus customer accounts, but no natural gas or coal. All are New Jersey monthly totals, not hourly Newark data, so they can only add slow trend context. We will source gas and coal series, or agree with Mr. Walker on substitutes, when we reach that stage.

B. Limitations

- *Weather knowledge.* Results with observed weather are better than a real forecaster could achieve. Typical weather is a rough stand-in for a real weather forecast, which we have not tested.
- *Short test windows.* Each horizon tests one stretch ending 31 May 2024; summer and winter peaks are untested.
- *Untuned model, few classes.* Tree settings are starting values. The tree has run on Res and GTI, the LSTM on Res only, out of 11 classes.
- *COVID window.* Excluding March 2020 to June 2021 removes about 13 months of a four-year series for nine classes.

- *Literature.* Section III rests on titles and snippets.

C. Next steps

1. Run all 11 classes with both models; report average error and the best and worst classes.
2. Use several test windows across seasons, and the same split, training data, and metrics for the tree and the LSTM.
3. Add customer counts (Weeks 6–9). Solar, gas, and coal series are future expansion after the initial model is complete.
4. Tune tree depth with time-ordered validation, then try random forest and gradient boosting.
5. Screen features with Spearman rank correlation and test rank-transformed inputs.
6. Extend the LSTM into a CRNN on one class and give it calendar inputs; compare with the trees.
7. Add figures: temperature against load, forecast against actual, error by hour and month.
8. Confirm the COVID window; compare exclusion with an adaptive method if time allows.

ACKNOWLEDGMENT

[Team: state who wrote which section, as in the proposal.]

AI USE DISCLOSURE

We used Claude, an AI assistant from Anthropic (Claude Sonnet 5 and Claude Sonnet 5.5, through `claude.ai` and Claude Code), in September and October 2026. This report was largely drafted by Claude from material gathered in that work; team members are responsible for checking it.

What Claude did. It read the course data and README files and summarized them. It wrote the decision-tree Python pipeline and ran it to produce the tree numbers here, and it found and fixed the mixed weather timestamp problem. It installed PyTorch in its own workspace, ran the teammate’s LSTM script unchanged on Residential, and scored the tree on the same period, including the typical-weather test. It ran the Google Scholar searches in a team member’s browser and wrote the Section III summaries from titles and snippets without reading the full papers. It also drafted a project paragraph, speaker notes, a notes PDF, a reading list, and this report.

What the team decided. Temperature as the top predictor and dates as the second signal, excluding COVID, resampling to hourly, pickling, rank-based methods, and a recurrent/convolutional stretch model came from the team’s notes and course material.

Limits. Paper descriptions may be wrong in details; the COVID dates and tree settings are working assumptions; reference lists are incomplete where Scholar cut off author names.

[Team: state who reviewed and re-ran the code, who read which papers, who wrote `newark_load_single.py` and whether AI helped (Claude ran it but could not tell), and any other AI use.]

REFERENCES

- [1] H. S. Hippert, C. E. Pedreira, et al., "Neural networks for short-term load forecasting: A review and evaluation," *IEEE Trans. Power Syst.*, 2001.
- [2] J. W. Taylor and P. E. McSharry, "Short-term load forecasting methods: An evaluation based on European data," *IEEE Trans. Power Syst.*, 2007.
- [3] W. Kong, Z. Y. Dong, Y. Jia, D. J. Hill, Y. Xu, et al., "Short-term residential load forecasting based on LSTM recurrent neural network," *IEEE Trans. Smart Grid*, 2017.
- [4] M. Abumohsen, A. Y. Owda, and M. Owda, "Electrical load forecasting using LSTM, GRU, and RNN algorithms," *Energies*, 2023.
- [5] M. Alhussein, K. Aurangzeb, and S. I. Haider, "Hybrid CNN-LSTM model for short-term individual household load forecasting," *IEEE Access*, 2020.
- [6] H. Eskandari, M. Imani, and M. P. Moghaddam, "Convolutional and recurrent neural network based model for short-term load forecasting," *Electr. Power Syst. Res.*, 2021.
- [7] K. Ullah, M. Ahsan, et al., "Short-term load forecasting: A comprehensive review and simulation study with CNN-LSTM hybrids approach," *IEEE Access*, 2024.
- [8] G. Dudek, "A comprehensive study of random forest for short-term load forecasting," *Energies*, 2022.
- [9] J. Moon, S. Rho, and S. W. Baik, "Toward explainable electrical load forecasting of buildings: A comparative study of tree-based ensemble methods with Shapley values," *Sustain. Energy Technol. Assess.*, 2022.
- [10] C. E. Asbury, "Weather load model for electric demand and energy forecasting," *IEEE Trans. Power App. Syst.*, 1975.
- [11] M. Imani, "Electrical load-temperature CNN for residential load forecasting," *Energy*, 2021.
- [12] Y. Wang, N. Zhang, and X. Chen, "A short-term residential load forecasting model based on LSTM recurrent neural network considering weather features," *Energies*, 2021.
- [13] K. Ijaz, Z. Hussain, J. Ahmad, S. F. Ali, M. Adnan, et al., "A novel temporal feature selection based LSTM model for electrical short-term load forecasting," *IEEE Access*, 2022.
- [14] B. Yu, J. Li, C. Liu, and B. Sun, "A novel short-term electrical load forecasting framework with intelligent feature engineering," *Appl. Energy*, 2022.
- [15] F. Alasali, K. Nusair, L. Alhmoud, and E. Zarour, "Impact of the COVID-19 pandemic on electricity demand and load forecasting," *Sustainability*, 2021.
- [16] D. Obst, J. De Vilmarrest, and Y. Goude, "Adaptive methods for short-term electricity load forecasting during COVID-19 lockdown in France," *IEEE Trans. Power Syst.*, 2021.
- [17] Course-provided dataset package "Data-Load_Prediction" (hourly load by rate class; Newark hourly weather; calendar flags; BEA, BLS, and EIA series), DS 340W.
- [18] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, 2011.
- [19] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019.

[Team: complete the references (full author lists, volume, pages, DOIs) in the format the course requires.]